

CareDiary Internship

Backend Development Assignment

Introduction

Congratulations on progressing to the next stage.

This assignment is designed to evaluate your backend development skills, including your expertise in Node.js, Express.js, TypeScript, PostgreSQL integration, Redis caching, message queues with RabbitMQ, WebSocket and SSE communication, and technical documentation.

Assignment Overview

Project: *Develop a "Live Health Status API Service" using Node.js, Express.js, TypeScript, PostgreSQL, Redis, and RabbitMQ.*

Project Requirements

1. Authentication API

- Implement token-based authentication (mocked; no OAuth required).
- Create a `/login` endpoint that accepts hardcoded email/password credentials and returns a mocked token.

2. Patient Health Record API (CRUD)

- **Database:** PostgreSQL.
- Implement the following REST APIs:
 - `POST /records` — Create a health record.
 - `GET /records/:id` — Retrieve a health record.
 - `PUT /records/:id` — Update a health record.
 - `DELETE /records/:id` — Delete a health record.
- **Fields:**

- `id` (UUID)
- `name` (string)
- `age` (integer)
- `status` (enum: Healthy, Sick, Critical)
- `last_updated` (timestamp)

3. Caching Layer (Redis)

- Implement caching for the `GET /records/:id` endpoint.
- Check Redis cache first; if not found, fetch from PostgreSQL and cache the result for 5 minutes.

4. Queue Processing (RabbitMQ)

- Set up a RabbitMQ queue named `health_updates`.
- Publish a message when a health record is created or updated.
- Consume messages from the queue and log them appropriately.

5. WebSocket Capability

- Set up a WebSocket server at `/ws`.
- Broadcast a notification to all connected clients whenever a health record is created or updated.

6. SSE-Based API (Server-Sent Events)

- Create an SSE endpoint at `/sse/health-updates`.
- Stream live health record updates to all connected clients.

7. Testing HTML Page (Frontend Tester)

- Create a simple HTML + JavaScript page to:
 - Login and receive a token.
 - Perform CRUD operations on health records.
 - Connect to WebSocket for real-time updates.
 - Connect to the SSE endpoint for streaming updates.
- (You may use AI tools to generate this HTML page.)

8. Documentation

- Provide a comprehensive `README.md` that includes:
 - Installation instructions (Node.js, PostgreSQL, Redis, RabbitMQ setup).
 - Environment variable setup guide.
 - Steps to run the server locally.
 - API usage examples with CURL commands.
 - Instructions to test WebSocket and SSE endpoints.

- Project structure and architectural decisions.
-

Additional Evaluation Points (Bonus)

- Set up ESLint and Prettier for code consistency.
 - Create a Docker Compose file to orchestrate PostgreSQL, Redis, and RabbitMQ services.
 - Maintain a modular codebase with separation of concerns.
-

Tech Stack

- Node.js (v18+ recommended)
 - Express.js
 - TypeScript
 - PostgreSQL (via `pg` or Prisma ORM)
 - Redis
 - RabbitMQ
 - Socket.io or native WebSocket
 - Server-Sent Events (SSE)
-

Submission Instructions

- Upload your code to a **public GitHub repository**.
 - Ensure a detailed `README.md` is included.
 - Share the GitHub repository link.
 - **Deadline:** *Sunday, 2 PM IST.*
-

Academic Integrity Policy

We use both automated tools and manual code reviews to detect AI-generated submissions.

You must complete this assignment independently. Submissions found to be heavily or fully AI-generated (excluding the HTML page) will be disqualified.

We value your ability to solve problems, design scalable backends, and communicate technical ideas clearly.

Evaluation Criteria

- Code Readability and Modularization
 - Proper Implementation of CRUD APIs
 - PostgreSQL Integration and Query Efficiency
 - Redis Caching Setup and Usage
 - RabbitMQ Queue Messaging
 - WebSocket and SSE Communication
 - Working Frontend Tester Page
 - Quality and Completeness of Documentation
 - Environment Configuration Management
 - TypeScript Best Practices
 - Error Handling and Validation
 - Bonus: Docker Compose Setup, ESLint + Prettier Configuration
-

Final Note

Good luck!

If you have any questions before or during the assignment, feel free to reach out.
